第4章　串（String）

4.1　串类型的定义

4.2　串的表示和实现

4.3　串的模式匹配算法

4.3　串的模式匹配算法

算法目的：确定主串中所含子串第一次出现的位置（定位）

定位问题称为串的模式匹配，典型函数为Index(S, T, pos)

算法种类：

- **BF算法**　（又称古典的、经典的、朴素的、穷举的）
- **KMP算法**

带回溯，速度慢

避免回溯，匹配速度快，
是全课程的亮点之一

BF算法的实现—即编写**Index(S, T, pos)**函数

例1： S=‘**ababcabcacbab**’，T=‘**abcac**’，pos=1，

求：串T在串S中第pos个字符之后的位置。

BF算法设计思想：

- 将主串S的第pos个字符和模式T的第1个字符比较，
  若相等，继续逐个比较后续字符；
  若不等，从主串S的下一字符（pos+1）起，重新与T第一个字符比较。
- 直到主串S的一个连续子串字符序列与模式T相等。返回值为S中与T匹配的子序列第一个字符的序号，即匹配成功。

否则，匹配失败，返回值 0。

利用演示系统看BF算法执行过程。

求子串位置的定位函数Index(S, T, pos) 的BF算法：

```
Int Index_BP(SString S, SString T, int pos) //见教材P79
{ //返回子串T在主串S中第pos个字符
0.
 // 其中，T非空，1≤pos≤StrLengt
   i=pos;      j=1;
  while ( i<=S[0] && j<=T[0] ) //如果i,j二指针在正常长度范围，
   {
     if (S[i] = = T[j] ) {++i, ++j; }   //则继续比较后续字符
     else {i=i-j+2; j=1;} //若不相等，指针后退重新开始匹配
   }
 if(j>T[0]) return i-T[0];  //T子串指针j正常到尾，说明匹配成功，
else return 0;        //否则属于i>S[0]情况，i先到尾就不正常
} //Index_BP
```

i-j=回到起始点之前，
i-j+1=回到首字符，
i-j+2=定位到第二个字符

BF算法的时间复杂度

讨论：

若n为主串长度，m为子串长度，则串的BF匹配算法最坏的情况下需要比较字符的总次数为 $(n-m+1)*m=O(n*m)$

最好的情况是：一配就中！ 只比较了m次。

最坏的情况是：主串前面n-m个位置都部分匹配到子串的最后一位，即这n-m位比较了m次，别忘了最后m位也各比较了一次，还要加上m！所以总次数为：$(n-m)*m+m=(n-m+1)*m$

一般的情况是：$O(n+m)$

推导方法：从最好到最坏情况统计总的比较次数，然后取平均

能否加快子串（又称模式串）的滑动速度?

能！利用已部分匹配过的信息，使主串S的指针i不必回溯，最坏情况也能达到$O(n+m)$

请看KMP算法！

KMP算法（特点：速度快）

① KMP算法设计思想

② KMP算法的推导过程

③ KMP算法的实现 （关键技术:计算next[j]）

④ KMP算法的时间复杂度

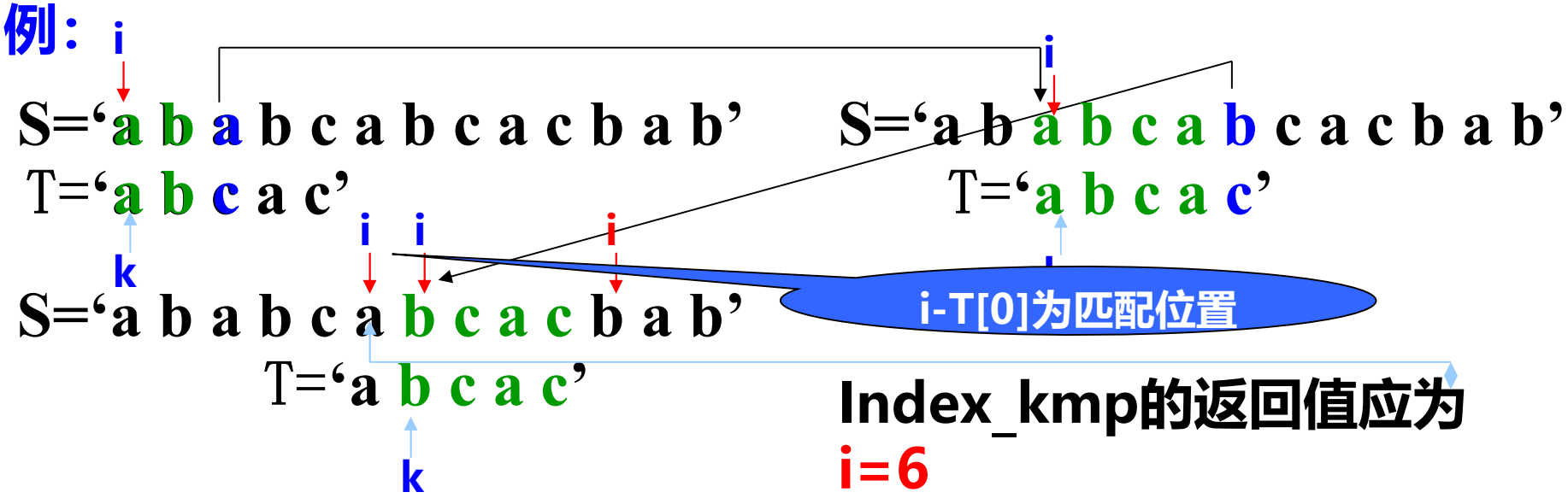
全书一大亮点!

利用演示系统看KMP算法的执行过程。

① KMP算法设计思想：（参见教材P80-84）

尽量利用已经部分匹配的结果信息，尽量让i不要回溯，加快模式串的滑动速度。

例：



Index_kmp的返回值应为 i=6

需要讨论两个问题：

①如何由当前部分匹配结果确定模式向右滑动的新比较起点k?

② 模式应该向右滑多远才是高效率的?

② KMP算法的推导过程：（见教材P81）

k是追求的新起点

请抓住部分匹配时的两个特征：

(1)

i

S=‘a b a b c a b c a c b a b’

T=‘a b c a c’

k

设目前打算与T的第k字符开始比较

则T的k-1～1位＝S前i-1～i-(k-1)位

即(4-2）式含义

‘$T_1 \ldots T_{k-1}$’

(2)

i

S=‘a b a b c a b c a c b a b’刚才肯定是在S的i处和T的第j字符 处失配

T=‘a b c a c’

k　　j

则T的j-1～j-(k-1)位＝ S前i-1～i-(k-1)位

即(4-3）式含义

‘$T_{j-(k-1)} \ldots T_{j-1}$’ 截取一段，但k有限制，1<k<j

两式联立可得： ‘$T_1 \ldots T_{k-1}$’=‘$T_{j-(k-1)} \ldots T_{j-1}$’

加速的前提：T首与Tj处有相同子串！

注意：j 为当前已知的失配位置，我们的目标是计算新起点 k。式中仅剩一个未知数k，理论上已可解！

奇妙的结论： k 仅与模式串T有关！

新起点 k怎么求？

根据模式串T的规律： $'T_1 \ldots T_{k-1}' = 'T_{j-(k-1)} \ldots T_{j-1}'$

由当前失配位置j(已知)，可以归纳出计算新起点 k的表达式。

令k = next[j] （k 与j 显然具有函数关系），则

$$next[j] = \begin{cases} 0 & \text{当 } j = 1 \text{ 时} \quad \text{//不比较} \\ \max\{k \mid 1<k<j \text{ 且 } 'T_1 \ldots T_{k-1}' = 'T_{j-(k-1)} \ldots T_{j-1}'\} \\ 1 & \text{其他情况} \end{cases}$$

取T首与Tj处最大的相同子串

讨论：

（1） next[j]有何物理意义?

（2） next[j]具体怎么求? —即KMP算法的实现

（1） next[j]有何物理意义？

T=‘a b a a b c a c’

$$next[j] = \max\{k \mid 1<k<j \text{ 且 } ‘T_1 \ldots T_{k-1}’ = ‘T_{j-(k-1)} \ldots T_{j-1}’\}$$

模式串从第1位往右直到K-1位

模式串从j的前一位往左经过K-1位

next[j]函数表征着模式T中最大相同前缀子串和后缀子串（真子串）的长度。

可见，模式中**相似部分越多，则next[j]函数越大**，它既表示模式T字符之间的相关度越高，也表示j位置以前与主串**部分匹配**的字符数越多。

即：next[j]越小，模式串初始位置j向右滑动得越远，与主串进行比较的次数越少，时间时间效率就越高。

（2） next[j]具体怎么求?

计算Next[j]的方法:

- **当j=1时，Next[j]=0;**

//Next[j]=0表示根本不进行字符比较

- **当j>1时，Next[j]的值为：模式串的位置从1到j-1构成的串中所出现的首尾相同的子串的最大长度加1。**

无首尾相同的子串时Next[j]的值为1。

// Next[j]=1表示从模式串头部开始进行字符比较

怎样计算模式T所有可能的失配点 j 所对应的 next[j]?

例：

模式串 T：	a	b	a	a	b	c	a	c
可能失配位 j：	1	2	3	4	5	6	7	8
新匹配位k=next[j]：	0	1	1	2	2	3	1	2

next[j]与s无关，可以预先计算

刚才已归纳：

$$\text{next}[j] = \begin{cases} 0 & \text{当 } j = 1 \text{ 时} \\ \max\{k \mid 1<k<j \text{ 且 } 'T_1 \ldots T_{k-1}' = 'T_{j-(k-1)} \ldots T_{j-1}'\} \\ 1 & \text{其他情况} \end{cases}$$

讨论：

j=1时, next[j]≡ 0；//属于“j=1”情况

j=2时, next[j]≡ 1；// 找不到1<k<j的k，属于“其他情况”

j=3时, k={2}，只需查看 ‘T_1’=‘T_2’成立否，No则属于其他情况

j=4时, k={2，3}，要查看 ‘T_1’=‘T_3’ 及 ‘T_1T_2’=‘T_2T_3’ 是否成立

j=5时, k={2，3，4}，要查看 ‘T_1’=‘T_4’， ‘T_1T_2’=‘T_3T_4’ 和 ‘$T_1T_2T_3$’=‘$T_2T_3T_4$’

以此类推，可得后续next[j]值。

从两头往中间比较

例（03级期末试题）：在KMP算法中，已知模式串为LIUYULIUWENYU，请写出模式串的next[j]函数值，并分析next[j]函数值的大小与KMP算法的比较次数多少（或时间）有何关系，简单解释你的理由。

答： next[j]函数值=0111112341111　**可用演示程序验证**

想一想：如果主串和模式均为二进制码流，用KMP算法效果如何?

再想一想：如果主串是外存中一个大文件，用KMP算法效果又如何?

下一个要讨论的问题是：如何用递推方式来求出最大相同子串的长度呢？换言之，如何让电脑替我们求出最大相同子串呢？这个问题一旦解决，整个KMP算法就可以掌握得很透彻了。

递推法编程，参见教材**P83**程序

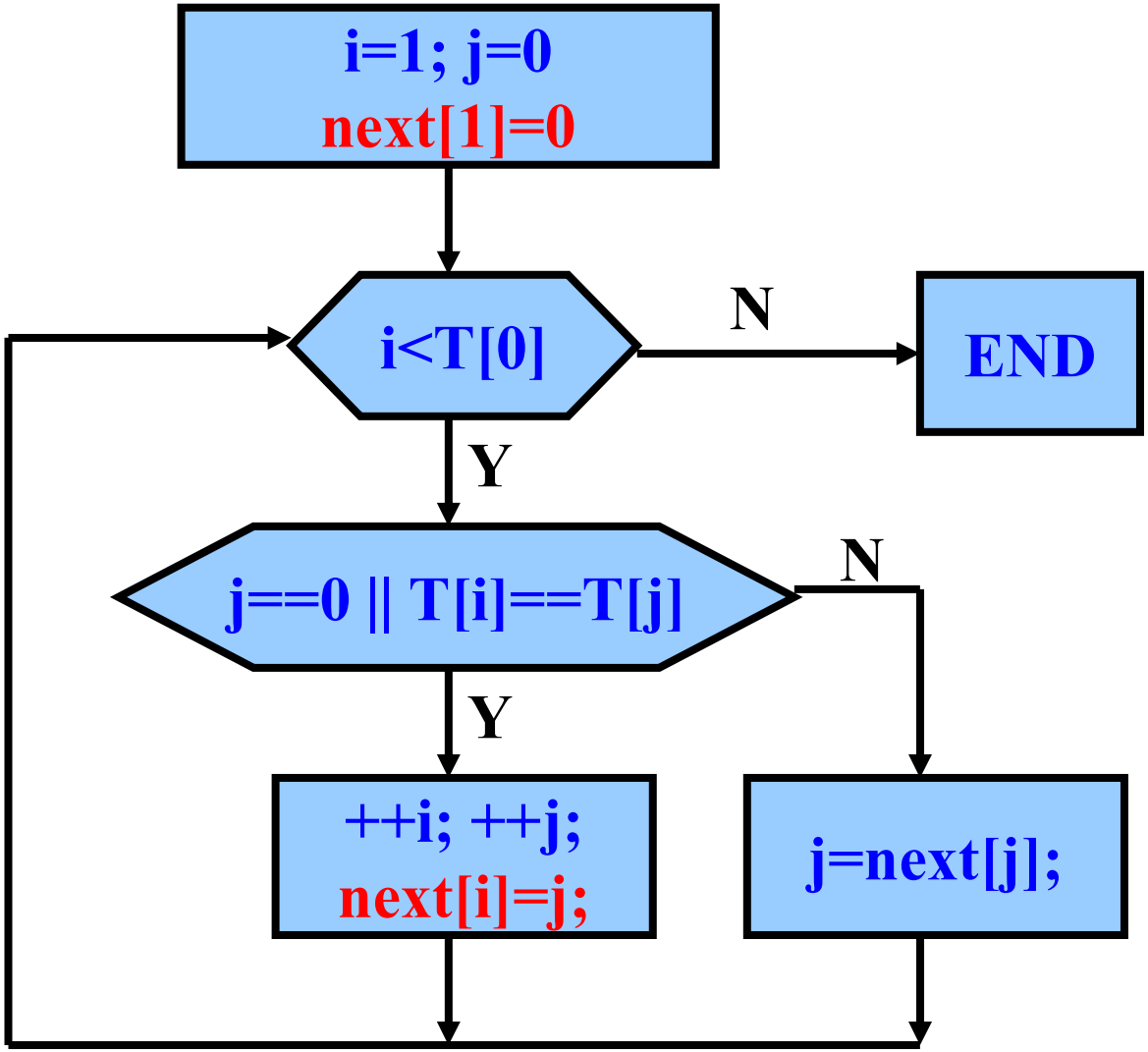
```
void get_next(SString T, int  &next[ ] ){
        //求模式串T的next函数值并存入数组next[ ]
i=1; j=0; next[1]=0;
while(i<T[0] ) //T[0]为T串之长度
 { if(j= = 0||T[i]= =T[j]){++i; ++j; next[i]=j;} //首次必j=0
    else j=next[j];
 }
}// get_next
```

	1	2	3	4	5	6	7	8
T[i]:	a	b	a	a	b	c	a	c
T[j]:		a	b	a	a	b	c	a	c
next[i]:	0	1	1	2	2	3	1	2

```
void get_next(SString T, int  &next[ ] ){
        //求模式串T的next函数值并存入数组next[ ]
i=1; j=0; next[1]=0;
while(i<T[0] ) //T[0]为T串之长度
 { if(j= = 0||T[i]= =T[j]){++i; ++j; next[i]=j;} //首次必j=0
    else j=next[j];
 }
}// get_next
```

	1	2	3	4	5	6	7	8	
T[i]:	a	b	a	a	b	c	a	c	
T[j]:		a	b	a	a	b	c	a	c
next[i]:	0	1	1	2	2	3	1	2	
i=	1	2	3	4	5	6	7	8	
j=	0	1	1	2	2	3	1	2	
		1		1		1			
						0			

求解next[j]流程图（递推）



补充：递推与递归的区别：

递推：由“小”到“大”递进
递归：由“大”到“小”嵌套

例如：求f(n)=n!

① 递推法：

```
fact=1;
for ( i=1; i<=n; i++)
fact*= i;
```

② 递归法：

```
long int fact(n)
        int n;
 {      long f;
         if(n>1)f=n*fact(n-1);
         else f=1;
        return(f);  }
```

③ KMP算法的实现—即Index( )操作的实现

第一步，先把模式T所有可能的失配点j 所对应的next[j]计算出来；

第二步：执行定位函数Index_kmp （与BF算法模块非常相似）

```
Int Index_KMP(SString S, SString T, int pos) { //见教材P82
  i=pos;      j=1;
  while ( i<=S[0] && j<=T[0] ) {
     if (j==0|| S[i] = = T[j] ) {++i, ++j}   //不失配则继续比较后续字符
    else {j=next[j];} //特点：S的i指针不回溯，而且从T的k位置开始匹配
     }
  if(j>T[0]) return i-T[0];  //子串结束，说明匹配成功
  else return0;
}//Index_KMP
```

④ KMP算法的时间复杂度

回顾BF的最恶劣情况：S与T之间存在大量的部分匹配，比较总次数为： $(n-m+1)*m=O(n*m)$

而KMP的情况是：由于指针i无须回溯，比较次数仅为n,即使加上计算next[j]时所用的比较次数m，比较总次数也仅为 $n+m=O(n+m)$**，大大快于BF算法。**

*注意：*由于BF算法在一般情况下的时间复杂度也近似于O(n+m)，所以至今仍被广泛采用。

KMP算法的用途：

因为主串指针i不必回溯，所以从外存输入文件时可以做到边读入、边查找，即“流水作业”！

讨论： next [j]是否完美无缺？

前面定义的next函数在某些情况下还是有缺陷的，
例如模式aaaab与主串aaabaaaab匹配时的情况：

先计算next[j]：

j：	1	2	3	4	5
T：	a	a	a	a	b
next[j]：	0	1	2	3	4

i:	1	2	3	4	5	6	7	8	9
S:	a	a	a	b	a	a	a	a	b
T:	a	a	a	a	b				

似乎慢了一点?
能否再提速?

先用演示程序验证

此时效率不高的原因为：子串前4位相同时，主串字符若与其中一个不相等，则不必缓移，而应将T首快速向右滑动。

由此派生出next函数的改进算法（不考）

next[j] 与 nextval[j]的区别何在？

T： a a a a b　　S: abcdaabcde

next[j]： 0 1 2 3 4

nextval[j]： 0 0 0 0 4

若子串前4位相同时，主串字符若与中间一个不相等，则T串应快速向右滑动，将起始位与下一主串字符比较。

next函数的改进算法见教材P84算法4.8，称为**nextval [j]**

```
void get_nextval(SString T, int  &nextval[ ] ){
        //next函数修正值存入数组nextval
i=1;  nextval[1]=0; j=0;
while(i<T[0] ){
   if(j= = 0||T[i]= =T[j] ){ ++i;++j;
If(T[i]!=T[j] ) nextval[i]=j;
else nextval[i]=nextval[j];  }
else j=nextval[j];  }
}// get_nextval
```

字符串的KMP算法小结：

目的：确定主串中所含子串第一次出现的位置

优点：避免主串指针回溯，匹配速度快，最坏情况也能达到 O(n+m)

与主串不回溯指针对应的模式T的新起点 k怎么求？

k= next[j] 或 nextval[j]

失配函数next[j]的物理意义：

模式T中最大相同前缀子串和后缀子串（真子串）的长度。

$$
\text{next}[\,j\,] = \begin{cases} 0 & \text{当 } j = 1 \text{ 时} \quad \text{//不比较} \\ \max\{\,k \mid 1<k<j \text{ 且 } `T_1 \ldots T_{k-1}' = `T_{j-(k-1)} \ldots T_{j-1}'\,\} \\ 1 & \text{其他情况} \end{cases}
$$

取T首与Tj处最大的相同子串

第4章小结

串
- 逻辑结构　s =‘ a_1a_2 ……..a_n’
- 存储结构
  - 定长顺序存储结构
  - 堆存储结构
  - 块链存储结构
- 操作(或运算)
  - 若干函数的实现
  - 模式匹配算法

模式匹配即子串定位运算，即如何实现 **Index(S,T,pos)**函数

BF算法———古典

KMP算法——快速（用**next[j]**或**nextval[j]**)

本章结束

下周周三交第4章作业：

严习题集的 **4.8　4.10　4. 30　4.31**共4题

复习C语言的指针：2011年百度笔试题目——解释下面ptr含义和不同

```
double* prt = &value;
const double* ptr = &value;
double* const ptr= &value;
const double* const ptr= &value;
```

2007百度笔试题：

编程1：找出字符串的最长不重复子串并输出其长度。实现时有无O（n）的算法？

编程2：找出一个字符串的最长回文子串，并使算法效率尽量高。

2013年百度笔试题：

字符串左移，void *pszStringRotate(char *pszString, int nCharsRotate),比如ABCDEFG，移3位变DEFGABC，要求空间复杂度O（1），时间复杂度O（n）

活学活用——计算字符串的相似度

- 许多程序会大量使用字符串。对于不同的字符串，我们希望能够有办法判断其相似程度。我们定义了一套操作方法来把两个不相同的字符串变得相同，具体的操作方法为：
- **1.** 修改一个字符（如把“**a**”替换为“**b**”）。
- **2.** 增加一个字符（如把“**abdd**”变为“**aebdd**”）。
- **3.** 删除一个字符（如把“**travelling**”变为“**traveling**”）。
- 比如，对于“**abcdefg**”和“**abcdef**”两个字符串来说，我们认为可以通过增加/减少一个“**g**”的方式来达到目的。上面的两种方案，都仅需要一次操作。把这个操作所需要的次数定义为两个字符串的距离，而相似度等于“距离**+1**”的倒数。也就是说，“**abcdefg**”和“**abcdef**”的距离为**1**，相似度为**1 / 2 = 0.5**。
- 给定任意两个字符串，你是否能写出一个算法来计算出它们的相似度呢？

第三章递归+第四章字符串

http://wenku.baidu.com/view/854b10df5022aaea998f0ffb.html